PHENIKAA AI HACKATHON 2026 · CHUNG KẾT · BÀI 1

Sơ đồ bài 1 Campus Courier

Bộ đồ nghề `KIT_BAI1` chạy từ thư mục dữ liệu BTC phát tới file nộp `predictions.json` như thế nào: mỗi cảnh đi qua ba khâu, script `tu_dong.py` sắp các bước theo thời gian, và phần nâng cao chọn mô hình cho từng robot.

Đọc ảnh (CV) · Đọc câu (NLP) · Chiến thuật robot · File nộp / quyết định

PHẦN 1

Một cảnh đi qua ba khâu

Mỗi dòng của `test/observations.json` gồm một ảnh bản đồ, một câu tiếng Việt giao nhiệm vụ và số hiệu robot. Đáp án là bước đi đầu tiên: `0 = UP, 1 = DOWN, 2 = LEFT, 3 = RIGHT`. Train và validation có kèm bản đồ ghi sẵn bằng chữ (`scenes.json`); test chỉ có ảnh, nên phải đọc ảnh.

ĐẦU VÀO 1 · ẢNH

Ảnh bản đồ campus 1152×864, bốn kiểu vẽ: `classic, night, print, sketch`. Có thể bị xoay.

ĐẦU VÀO 2 · CÂU + ROBOT

Ví dụ: *“Huỷ đơn giao nhà xe. Thay vào đó… nhớ ghé lớp học lấy tập bài kiểm tra.”* Có câu không dấu, câu đính chính, câu lan man.

↓ ↓

KHÂU 1 · ĐỌC ẢNH

`src/run_cv.py` → `src/cv_infer.py` : bộ dò quét ảnh, khớp lưới (kể cả ảnh xoay), đọc chú giải màu của từng ảnh.

Mạng MLP (`.npz`) và CNN (`node_cnn` , `edge_cnn` , `swatch_cnn` , file `.onnx`) phân loại từng giao lộ và đoạn đường.

Ra: đồ thị lưới gồm đoạn đường (thường / đông / có mái / đóng), bậc thang, một chiều, địa điểm, vị trí và hướng mũi của robot, thời tiết.

KHÂU 2 · ĐỌC CÂU

`src/nlp3.py` (bộ phân tích câu, học lại trên dữ liệu mới) + mô hình nhúng câu `multilingual-e5-small` (`outputs/e5_small` , chạy bằng onnx).

Hiểu được đính chính (“huỷ…, thay vào đó…”), câu không dấu, địa điểm tả qua bản đồ (“phía bắc”, “gần nhất”).

Ra: đích, điểm ghé (via), cờ gấp / dễ vỡ và các cờ mới nếu đề thêm.

↓ ghép bản đồ + nhiệm vụ ↓

KHÂU 3 · CHIẾN THUẬT

Với mỗi hướng đi có thể từ chỗ robot đứng, `src/strat_ml.py` (`move_feats`) và `nen.py` (`scene_feats`) tính đặc trưng: đi hướng đó thì đường còn lại dài bao nhiêu, qua bao nhiêu đoạn đông / có mái / bậc thang, có gần đích hay điểm ghé không, trời đêm hay mưa...

UP	DOWN	LEFT	RIGHT
đặc trưng	đặc trưng	đặc trưng	đặc trưng

Mỗi robot có thói quen riêng mà đề chỉ gợi ý mơ hồ trong `robots.json` (vd "chỉ muốn tới nơi bằng ít đoạn đường nhất"). Vì vậy **mỗi robot một mô hình riêng** học từ train: bản nền dùng HistGradientBoosting, bản nâng cao thử thêm vài cách (Phần 3). Mô hình chấm điểm bốn hướng và chọn hướng điểm cao nhất; hướng bị tường chặn bị loại.

↓

ĐẦU RA

Một số 0–3 cho mỗi dòng test, theo đúng thứ tự `test/observations.json` , ghi thành danh sách JSON: `runs/chungket/predictions.json` .

PHẦN 2

Thứ tự chạy của `tu_dong.py`

Bấm `CHAY_BAI1.bat` (máy Windows) hoặc `bash JUPYTER/bai1.sh` (JupyterHub) đều gọi `python tu_dong.py <dữ liệu> chungket`. Việc đọc ảnh chạy nền ở làn phải trong lúc làn giữa học trên train. Thời gian đo trên máy 8 nhân với dữ liệu vòng private; bước nào đã xong thì lần chạy sau bỏ qua.

PHÚT	LÀN CHÍNH	LÀN NỀN
0	**0 · soi đề mới** So cấu trúc dữ liệu với vòng private. Dòng **KHÁC** cho biết BTC nâng cấp ở đâu. `kiem_tra_de.py`	**1 · đọc ảnh validation, rồi test** Chạy song song nhiều tiến trình (biến `PROCS`, tối đa 8 trên JupyterHub). validation ~4–7 phút test ~16 phút trở lên (1.200 ảnh, chưa đo trên đề mới) `src/run_cv.py → cache/world_*.pkl, det_*.pkl` → kết quả được giữ lại khi HỌC LẠI
0	**0b · soi dữ liệu đầu vào** Tìm ảnh hỏng, câu rỗng; so đặc trưng train / validation / test. Dòng **LỆCH** = test khác validation, nên điểm web sẽ thấp hơn điểm tự chấm. `soi_du_lieu.py`	
1	**2a · kho câu gấp, dễ vỡ** `train/sent_inventory.py`	
2–6	**2b, 2c · học bộ đọc câu + mô hình chiến thuật** 2b học chỉ trên train (để tự chấm trên validation). 2c học trên train + validation (để nộp). `nen.py fit trainonly · nen.py fit final`	
~8	**3 · bảng CHẨN ĐOÁN** Chờ đọc ảnh validation xong rồi chấm từng khâu (Phần 4). `nen.py chandoan → outputs/chan_doan.json`	
30–40	**4 · FILE NỘP bản nền → nộp ngay** Chờ đọc ảnh test xong, đoán toàn bộ test. In điểm tự chấm, ghi một dòng vào sổ điểm. `nen.py test → predictions.json, predictions_ban_nen.json, nhat_ky_diem.txt`	

+20–45

5 · nâng cao
Học mô hình mạnh hơn cho từng robot (Phần 3). Chỉ chạy khi có đúng 10 robot 0–9 và có torch; lỗi thì bỏ qua, bản nền vẫn còn.

`nang_cao.py → outputs/chon.json`

6 · chẩn đoán lại

`nen.py chandoan`

~65–90

7 · FILE NỘP bản nâng cao, tự so với bản nền
Điểm tự chấm hơn bản nền ít nhất 0,003 thì in **TỐT HƠN → NỘP**. Không hơn thì in **TRẢ VỀ BẢN NỀN** và chép bản nền đè lại vào `predictions.json`; đã nộp bản nền thì khỏi nộp nữa.

`predictions.json`,
`predictions_nang_cao.json`,
`predictions_HHMM.json`

PHẦN 3

Bên trong bước nâng cao

`nang_cao.py` thử thêm ba loại mô hình cho từng robot, rồi giữ cho mỗi robot cái nào đúng nhất trên validation. Robot nào mô hình mới không hơn thì giữ mô hình nền.

A · LIỆT KÊ ĐƯỜNG TỐT

`train/pareto.py`: với mỗi cảnh, liệt kê các đường đi "không thua kém nhau" (Pareto) theo độ dài, số đoạn đông, có mái, bậc thang... ~2–3 phút.

B · SIÊU TUYẾN TÍNH

`train/hyper_train.py` (cần torch): mỗi robot học trọng số chấm điểm các đường. Hai biến thể `full` và `goal`, mỗi biến thể 2 hạt giống, cộng thêm bản trung bình của hai. ~20 phút.

C · LUẬT THAM LAM

`train/greedy_rules.py`: cho robot 9 ("chỉ nhìn một bước"), dò luật chọn bước kế tiếp tốt nhất. Vài giây.

Chọn: `nen.py chon` chấm mọi ứng viên của từng robot trên validation (nền HGB, siêu tuyến tính full, goal, trung bình, tham lam) và ghi lựa chọn vào `outputs/chon.json`. Bước 7 dùng file này để đoán test.

PHẦN 4

Đọc bảng CHẨN ĐOÁN và điểm số

Bảng chấm cùng một tập validation theo bốn cách, để biết điểm mất ở khâu nào. Dòng (4) là điểm thật; so (4) với (2) và (3) để thấy khâu đọc ảnh hay khâu đọc câu làm rơi điểm.

DÒNG	BẢN ĐỒ DÙNG	CÂU DÙNG	CHO BIẾT
(1)	đúng (scenes.json)	đúng (nhãn)	trần của `chiến thuật`: nếu đọc ảnh và câu không sai gì
(2)	đúng	máy đọc	(1) − (2) = điểm mất do `đọc câu`
(3)	máy đọc	đúng	(1) − (3) = điểm mất do `đọc ảnh`
(4)	máy đọc	máy đọc	`điểm thật`, gần với điểm web nhất

ĐÃ ĐO TRÊN DỮ LIỆU VÒNG PRIVATE	ĐIỂM TỰ CHẤM (VALIDATION, MACRO ACCURACY)
baseline `starter.py` của BTC (hướng hay gặp nhất của mỗi robot)	≈ 0,25
chỉ đọc câu, không đọc ảnh (TF-IDF, mỗi robot một mô hình)	0,26
bản nền (bước 4)	0,79
bản nâng cao (bước 7)	0,90
bản làm tay của MinnToet ở vòng loại · điểm web	0,93 · 0,83

Điểm web thường thấp hơn điểm tự chấm khoảng 7–10 điểm vì test khác validation (vòng private: câu test dài hơn, 41% không dấu so với 34%). Hai dòng đầu cho thấy gần như toàn bộ điểm nằm ở việc đọc được bản đồ từ ảnh.

PHẦN 5

Kết quả nằm ở đâu

```
KIT_BAI1/runs/
├── du_lieu_chungket.txt      # đường dẫn dữ liệu, để HOC_LAI dùng lại
├── man_hinh_chungket.txt     # (JupyterHub) mọi chữ hiện trên màn hình
└── chungket/
    ├── predictions.json          # FILE NỘP: luôn là bản script khuyên nộp
    ├── predictions_ban_nen.json
    ├── predictions_nang_cao.json
    ├── predictions_1432.json     # mỗi lần ra file nộp lưu thêm một bản có giờ
    ├── nhat_ky_diem.txt          # giờ tạo · bản nền/nâng cao · điểm tự chấm · tên file
    ├── log/                      # log từng bước; lỗi thì đọc dòng cuối
    ├── cache/                    # kết quả đọc ảnh (world_*, det_*), đặc trưng
    └── outputs/                  # mô hình đã học, chan_doan.json, chon.json
```

PHẦN 6

Khi đề có chỗ mới: sửa rồi học lại

1

Thấy vấn đề

Dòng KHÁC lạ, `!!! BƯỚC LỖI`, hoặc dòng nào đó trong bảng chẩn đoán thấp.

2

Hỏi AI của BTC

Dán 20 dòng cuối của log + đúng hàm liên quan (mẫu A, B, C trong sổ tay).

3

Sao lưu, dán hàm mới

Copy file trước khi sửa; dán đè đúng hàm; lưu.

4

Học lại

`HOC_LAI.bat` hoặc `bash JUPYTER/hoc_lai.sh`: xoá phần đã học, giữ kết quả đọc ảnh, chạy lại từ bước 2.

5

So sổ điểm

Dòng mới trong `nhat_ky_diem.txt` cao hơn file đã nộp ít nhất 0,01 thì nộp.

Các hàm hay phải sửa khi đề đổi định dạng: `true_legs`, `norm_for_parser`, `scene_feats` trong `nen.py`; `move_feats` trong `src/strat_ml.py`. Đề đổi kiểu vẽ bản đồ hay thêm loại địa điểm thì khâu đọc ảnh (`src/cv_infer.py`) có thể sai, và đó là phần khó sửa nhất.

PHẦN 7

Hai nơi chạy, cùng một bộ code

VIỆC	MÁY WINDOWS CỦA ĐỘI	JUPYTERHUB (TERMINAL)
đưa code lên	đã có sẵn	`git clone` repo B1, hoặc upload `kit.zip` rồi giải nén
cài đặt	đã cài trong `.venv`	`bash ~/B1/JUPYTER/cai_dat.sh` (tự tải lại model 470 MB nếu thiếu)
chạy bài 1	bấm đúp `CHAY_BAI1.bat`, kéo thả thư mục	`bash ~/B1/JUPYTER/bai1.sh <Copy Path>`
học lại	`HOC_LAI.bat`	`bash ~/B1/JUPYTER/hoc_lai.sh`
lấy file nộp	mở thẳng thư mục `runs\chungket`	chuột phải `predictions.json` → Download

---

Nguồn: code trong `KIT_BAI1` (gốc là lời giải vòng private COURIER2 của MinnToet, thêm phần chạy tự động) và số đo trên dữ liệu vòng private. Hướng dẫn từng bước: `HUONG_DAN_THI.docx`.